A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

```
docker save busybox | gzip > busybox.tar.gz
docker load < busybox.tar.gz
```

- **ROS (1) version: Kinetic Kame**

Kinetic Kame (May 2016 - May 2021)

Required Support for:

- Ubuntu Wily (15.10)
- Ubuntu Xenial (16.04)

Recommended Support for:

- Debian Jessie
- Fedora 23
- Fedora 24

Minimum Requirements:

- C++11
  - GCC 4.9 on Linux, as it's the version that Debian Jessie ships with
- Python 2.7
  - Python 3.4 not required, but testing against it is recommended
- Lisp SBCL 1.2.4
- CMake 3.0.2
  - Debian Jessie ships with CMake 3.0.2
- Boost 1.55
  - Debian Jessie ships with Boost 1.55

Exact or Series Requirements:

- Ogre3D 1.9.x
- Gazebo 7
- PCL 1.7.x
- OpenCV 3.x
- Qt 5.3.x
- PyQt5

Build System Support:

- Same as Indigo

Install guide: [kinetic/Installation/Ubuntu - ROS Wiki]

[ROS Tutorial: Communication between two Computers - Machine Learning Site]
https://docs.ros.org/en/foxy/How-To-Guides/Run-2-nodes-in-single-or-separate-docker-containers.html

ALL STEPS ARE ALREADY DOCUMENTED FOR ROS1

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

ROS INSTALLATION version: Kinetic Kime (for Ubuntu 16.04)

Follow steps at https://wiki.ros.org/kinetic/Installation/Ubuntu
Then https://wiki.ros.org/ROS/Tutorials/InstallingandConfiguringROSEnvironment
Steps explained below in a successive way.

*INSTALLATION

Setup sources.list

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

Setup keys with curl

```
sudo apt install curl # if you haven't already installed curl
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -
```

Make sure package index is up to date

```
sudo apt-get update
```

Install using apt-get

```
sudo apt-get install ros-kinetic-desktop-full
```

*SETUP ENVIRONMENT

Echo setup to bash file and source automatically to it

```
echo "source /opt/ros/kinetic/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Useful dependencies for ros

```
sudo apt install python-rosdep python-rosinstall python-rosinstall-generator python-wstool build-essential
```

Init rosdep

```
sudo rosdep init
rosdep update
```

*CREATE WORKSPACE WITH Catkin:

Create and build ws

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/
catkin_make
source devel/setup.bash                    (?)
```

*COMMUNICATE THROUGH IP ROS1->ROS1, PC A is denoted as A, PC B as B | https://machinelearningsite.com/ros-communication-between-two-computers/#ros-1-ros-1-communication

A

```
export ROS_MASTER_URI=https://192.168.0.1:11311 # ip of computer A
export ROS_IP=192.168.0.1
```

```
# on Computer B
export ROS_MASTER_URI=https://192.168.0.1:11311
```

These variables are limited and recognized only in the terminal they are declared in. To make them global,
write declare them directly into the bash script and source your bash script to update the environment.
This way, everytime you open a terminal, the environment will declare those variables automatically.

```
# on Computer A
echo "export ROS_MASTER_URI=https://192.168.0.1:11311" >> ~/.bashrc          MASTER IP
echo "export ROS_IP=192.168.0.1" >> ~/.bashrc                                YOUR IP
source ~/.bashrc
```

Add export lines to bash file

```
# on Computer B
echo "export ROS_MASTER_URI=https://192.168.0.1:11311" >> ~/.bashrc          MASTER IP
echo "export ROS_IP=192.168.0.2" >> ~/.bashrc                                YOUR IP
source ~/.bashrc
```

Run master node

```
# on computer A
roscore
```

Run script "talker" from "roscpp_tutorials" package

```
# open a different terminal on computer A
rosrun roscpp_tutorials talker
```

Check if B can reach master node

```
# on computer B
rostopic list
```

Run file "listener" in "roscpp_tutorials" package

```
rosrun roscpp_tutorials listener
```

Inside your workspace, create editable packages using catkin:

```
catkin_create_pkg first_pkg std_msgs roscpp                    Where "first_pkg" is the package name
```

*Then add your cpp files for communication (listener and talker scripts) inside /first_pkg/src

*Also add the following lines to CMakeLists.txt INSIDE "first_pkg/src" folder

```
add_executable(nodename src/scriptname.cpp)
target_link_libraries(nodename ${catkin_LIBRARIES})

add_executable(nodename2 src/scriptname2.cpp)
```

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

target_link_libraries(nodename2 ${catkin_LIBRARIES})

MAKE SURE "ROS_PACKAGE_PATH" IS SET TO WHERE THE "first_pkg" IS. you can see where the path is located by : $env | grep ROS
if not, move "first_pkg" to that location

Add the sourcing for the location of the catkin bash file to the common bashrc file
echo "source devel/setup.bash" >> ~/.bashrc
And source to bashrc again
source ~/.bashrc

Now you can run $roscore in one terminal as master
and $rosrun first_pkg talker to run your own "talker" node
etc..

D*OCKER ROS2 COMMUNICATION DOCUMENTATION*

Try see if docker is already on your system
$docker

if not installed
$sudo apt install docker

Pull ros-humble using docker
$docker pull osrf/ros:humble-desktop

Create docker container for ros2 (humble)
$docker run -it osrf/ros:humble-desktop

Find active containers (and all created containers using flag: -a)
$docker ps

Create docker container using the host name as id and network communication capabilities
$docker run -it --network host osrf/ros:humble-desktop

Remove all exited containers and its volume on the machine (Root required)
$docker rm -v $(docker ps --filter status=exited -q)

$docker start 6877d98dd7ad
$docker exec -it 6877d98dd7ad /bin/bash (container id with volume, called testVolume)

To be able to communicate through network, we need to set the domain ID inside the container
$export ROS_DOMAIN_ID=<id_int>

and to run HelloWorld communication through ros2:
$ros2 run demo_nodes_cpp talker

and on the other machine (on the same network)
$ros2 run demo_nodes_cpp listener
#Creating Docker file
Docker file helps in building an docker image which gives more control over the image. Think of it as a blue print for image.

Creating Dockerfile:

In Linux run command to create docker file: touch Dockerfile
then write the following to docker (this example is for raspberrypi)

```
# Use Ubuntu 20.04 (Focal) as a base image
FROM ubuntu:focal

# Set environment variables
ENV LANG=C.UTF-8 LC_ALL=C.UTF-8
ENV ROS_DISTRO=foxy
ENV DEBIAN_FRONTEND=noninteractive

# Update and install necessary packages
RUN apt-get update && apt-get install -y \
    locales \
    curl \
    gnupg2 \
    lsb-release \
    build-essential \
    && locale-gen en_US en_US.UTF-8 \
    && update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

# Add the ROS 2 apt repository for Foxy
RUN curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | apt-key add -
\
    && sh -c 'echo "deb [arch=arm64] http://packages.ros.org/ros2/ubuntu $(lsb_release -cs)
main" > /etc/apt/sources.list.d/ros2-latest.list'

# Install ROS 2 Foxy desktop packages
RUN apt-get update && apt-get install -y \
    ros-dev-tools \
    ros-foxy-desktop \
    python3-argcomplete \
    python3-colcon-common-extensions \
    && apt-get clean

# Source the ROS 2 setup script automatically
RUN echo "source /opt/ros/foxy/setup.bash" >> ~/.bashrc


# Set the entrypoint to bash for interactive mode
```

ENTRYPOINT ["/bin/bash"]

to build this go to same folder as dockerfile and run: docker build -t image_name .

Docker Volumes:
Docker does not save anything

Create work space
Creating a workspace — ROS 2 Documentation: Foxy documentation
Create editable ros files
https://docs.ros.org/en/foxy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html
Create image subscriber
image_transport/Tutorials/SubscribingToImages - ROS Wiki

rclcpp docs
rclcpp: rclcpp: ROS Client Library for C++ (ros2.org)

px4 install w gazebo link
https://docs.px4.io/main/en/ros2/user_guide.html#humble
useful for r-pi px4 connectivity
https://docs.px4.io/main/en/middleware/uxrce_dds.html

docker volume create `testVolume`

```
sudo docker run -it --network host -v testVolume:/home
[osrf/ros:foxy-desktop]
```

Image library:
https://github.com/ros-perception/image_transport_tutorials

Camera software - Raspberry Pi Documentation

Simulation setup, using px4, humble in gazebo
Create a docker container using the Dockerfile from GitHub

```
From the location of Dockerfile folder run
$docker build -t .

Note: commit docker container once your environment is setup to save
the contents

$docker commit <container_id> <name_of_new_container>

Follow the following link for additional information regarding the
px4 hardware agent setup:
https://docs.px4.io/main/en/ros2/user_guide.html

Download PX4-autopilot from git and make
$cd
$git clone https://github.com/PX4/PX4-Autopilot.git --recursive
$bash ./PX4-Autopilot/Tools/setup/ubuntu.sh
$cd PX4-Autopilot/
$make px4_sitl

Install ros2 humble and set configurations
$sudo apt update && sudo apt install locales
$sudo locale-gen en_US en_US.UTF-8
$sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
$export LANG=en_US.UTF-8
$sudo apt install software-properties-common
$sudo add-apt-repository universe
$sudo apt update && sudo apt install curl -y
$sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
$echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list
> /dev/null
$sudo apt update && sudo apt upgrade -y
$sudo apt install ros-humble-desktop
$sudo apt install ros-dev-tools
$source /opt/ros/humble/setup.bash && echo "source
/opt/ros/humble/setup.bash" >> .bashrc
```

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

```
Install required pyros dependencies
pip install --user -U empy==3.3.4 pyros-genmsg setuptools

Set up agent inside docker container
$git clone https://github.com/eProsima/Micro-XRCE-DDS-Agent.git
$cd Micro-XRCE-DDS-Agent
$mkdir build
$cd build
$cmake ..
$make
$sudo make install
$sudo ldconfig /usr/local/lib/

Start the agent that communicates with the simulated px4 client
$MicroXRCEAgent udp4 -p 8888

Test to see if simulation is correctly installed and works
$make px4_sitl gz_x500

Follow the steps in this link
https://github.com/ARK-Electronics/tracktor-beam/tree/aruco_tutorial

THE FOLLOWING IS ALL STEPS WHEN USING PREBUILT DOCKER IMAGE (~12gb)




Download docker image
then import:
$docker import gazebohumblev7.tar gazebohumblev1

Start docker container using image "gazebohumblev1" with connected
display
$xhost +local:docker
$docker run -it --network host -e DISPLAY=:0 -v
/tmp/.X11-unix:/tmp/.X11-unix gazebohumblev1

To show docker containers
$docker ps
to open in multiple terminals use
```

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

```
$docker exec -it <container_id> bash

Start px4 on gazebo
$pkill -9 ruby
$unset GZ_IP
$unset GZ_PARTITION

$make px4_sitl gz_x500_mono_cam_down_aruco

Run agent
$MicroXRCEAgent udp4 -p 8888

Run bridge for a topic (image)
$ros2 run ros_gz_bridge parameter_bridge
/camera@sensor_msgs/msg/Image@gz.msgs.Image
/camera_info@sensor_msgs/msg/CameraInfo@gz.msgs.CameraInfo

(camera config topic)
$ros2 run ros_gz_bridge parameter_bridge
/camera_info@sensor_msgs/msg/CameraInfo@gz.msgs.CameraInfo

login into qgcuser to open qgc software
$su qgcuser     pwd= 123

extract files from appimage
$cd
$./QGroundControl.AppImage --appimage-extract    (do this once)

go inside squashfs-root and run file  (always run from folder)
$cd squashfs-root
$./QGroundControl

run aruco tracker pkg
$ros2 run aruco_tracker aruco_tracker

run camera view pkg
$ros2 run rqt_image_view rqt_image_view
```

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

```
#when you run
$docker swarm init
#it creates the current node as a master and gives below command to
add other worker nodes

#port busy issues? run:

$sudo systemctl stop docker

then run swarm init again

$docker swarm join --token
SWMTKN-1-205scmki363jz6m31bvvx6vs5jpq20foskh5s0qw0j37bmkkml-dzi4dr5j
hg63quc1fuz9r4kr7 10.132.134.218:2377
#to list nodes in docker swarm
$docker node ls



#to deploy stack
$docker stack deploy -c docker-compose.yml my_stack
#to stop stacks in case of issue
$docker service stop  my_stack_my_service
or
$docker stack rm my_stack



to view status of your service
$docker service ps  my_stack_my_service

docker-compose.yml

services:
  my_service:
    image: v2
    deploy:
      placement:
        constraints:
          - "node.id == r0n3f93c8e5vtmv5pxryqtlv9"
    entrypoint: ["/bin/bash"]
    tty: true

#Whenever you're done running a node in Raspberry Pi
docker swarm leave
```

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

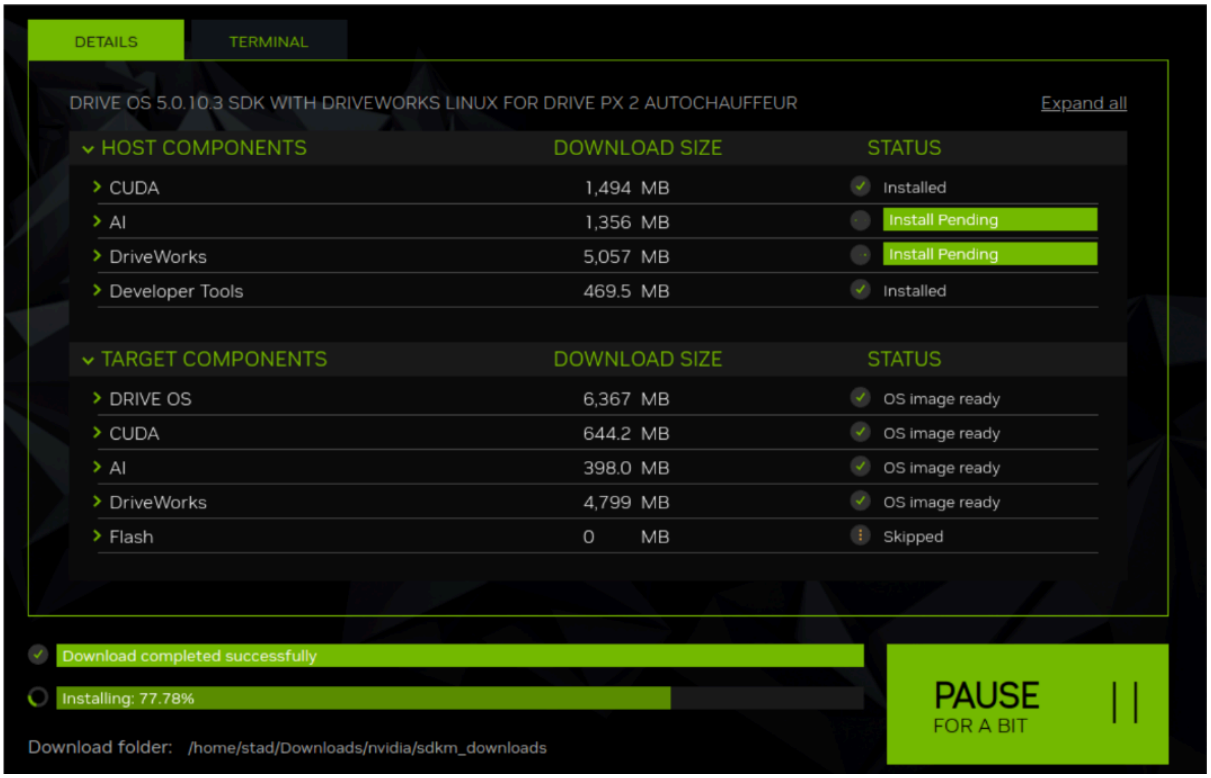
When access to TLS isnt working on the px2, run
$ export LD_PRELOAD=/usr/lib/aarch64-linux-gnu/libgomp.so.1

PX2 Reflash:
Log in to host PC (Provided by Martin) with password and username: stad
run command "sdkmanager" in the terminal.
when you run it, will give you default location of sdkmanager, (if not use: /home/stad/Downloads/nvidia/sdkm_downloads) install it and proceed.
then it will give you GUI like below

Connect PX2 and the host PC with USB (the port right above the HDMI on PX2).

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project

after this install everything as below



when this is done will give you the option to flash then flash it and you are good to go.

A document to guide other developers on ROS1&2 and Docker, with example commands. Specifically on MDU's STAD Project